

Data Types & Table Design

Day 2 • Focus: Proper table design + professional database structure.



SQL MASTERY

What is a Data Type?

Data Type means: What kind of data a column can store.

If you choose the wrong data type:

- ✗ Data becomes messy
- ✗ Performance issues arise
- ✗ Future schema problems

Correct data type = Clean database.



Common Data Types (Part 1)



1. INT (Integer)

Used for whole numbers. Ideal for counts, identifiers, and ages.

Examples:

- ✓ Age → 18
- ✓ Student ID → 101

```
age INT;
```



2. VARCHAR

Used for short text. Defines a maximum allowed character length.

Examples:

- ✓ Name → "Rahul"
- ✓ City → "Delhi"

```
name VARCHAR(50);
```



3. TEXT

Used for long text when content length may be large and unpredictable.

Examples:

- ✓ Description
- ✓ Address / Comments

```
description TEXT;
```


Common Data Types (Part 2)

4. FLOAT / DECIMAL

Used for storing numbers with decimals.

Examples:

✓ Price → 199.99

✓ Salary → 25000.50

** DECIMAL is preferred for currency to avoid rounding errors.*

```
marks FLOAT; salary DECIMAL(10,2);
```

5. DATE

Used exclusively for storing dates in the standard format.

Format: YYYY-MM-DD

Example:

✓ Date of Birth → 2026-02-23

```
dob DATE;
```




Database Constraints

Constraints control how data behaves. They enforce rules at the database level to ensure your database remains strong, reliable, and error-free.

Core Constraints



PRIMARY KEY

Uniquely identifies each row in a table.

- Cannot be duplicate
- Cannot be NULL

Ex: Student ID should never repeat.

```
id INT PRIMARY  
KEY;
```



NOT NULL

Enforces that a column must always have a value.

Ex: A student's Name should never be empty.

```
name VARCHAR(50)  
NOT NULL;
```



AUTO_INCREMENT

Automatically increases the numeric value sequentially (1, 2, 3, 4...).

You don't need to insert the ID manually.

```
id INT PRIMARY KEY  
AUTO_INCREMENT;
```




Hour 2 – Practical

Transitioning from theory to action: Writing real SQL queries to structure tables and manage database records.

Creating the Environment

Step 1 – Create Database

```
CREATE DATABASE school_db; USE school_db;
```

Step 2 – Create Professional Table

This is a properly designed table using appropriate data types and essential constraints.

```
CREATE TABLE students ( id INT PRIMARY KEY  
AUTO_INCREMENT, name VARCHAR(50) NOT NULL, age INT,  
city VARCHAR(50), marks FLOAT );
```



Step 3 & 4 – Insert and View Data

Inserting Records

Notice that we omit the `id` column. `AUTO_INCREMENT` handles it automatically.

```
INSERT INTO students (name, age, city, marks) VALUES  
( 'Rahul', 18, 'Delhi', 85.5); INSERT INTO students (name, age,  
city, marks) VALUES ( 'Amit', 19, 'Mumbai', 90.0); INSERT INTO  
students (name, age, city, marks) VALUES ( 'Priya', 17, 'Delhi',  
88.2);
```

Retrieving Data

To view all columns and rows in our newly populated table:

```
SELECT * FROM students;
```

i The output will show IDs generated sequentially (1, 2, 3) because of the `AUTO_INCREMENT` constraint.

Query Result: SELECT * FROM students;

ID (PK)	NAME	AGE	CITY	MARKS
1	Rahul	18	Delhi	85.5
2	Amit	19	Mumbai	90.0
3	Priya	17	Delhi	88.2

The database automatically generated the IDs ensuring no duplication.

Behind the Scenes

⚙️ How MySQL Handles It

Because you applied **PRIMARY KEY** and **AUTO_INCREMENT**, MySQL automatically takes over the heavy lifting.

It executes background checks upon every **INSERT** to validate the schema rules before committing the data.

✅ Professional Systems

This automated handling accomplishes three critical things:

- ✅ Ensures row uniqueness
- ✅ Generates sequential IDs
- ✅ Prevents duplicate data errors

That's exactly how real-world enterprise systems work.



Create Database: Execute the command to create `college_db`.



Create Table: Build a table named `teachers` with the following columns:

- `id` (INT, Primary Key, Auto Increment)
- `name` (VARCHAR, NOT NULL)
- `subject` (VARCHAR)
- `salary` (DECIMAL)



Insert Data: Write INSERT queries to add 5 distinct teachers.



Verify: Run a SELECT statement to show all teachers.

Do this without copying previous code!

Important Concept



**Good database design equals
fewer future problems.**

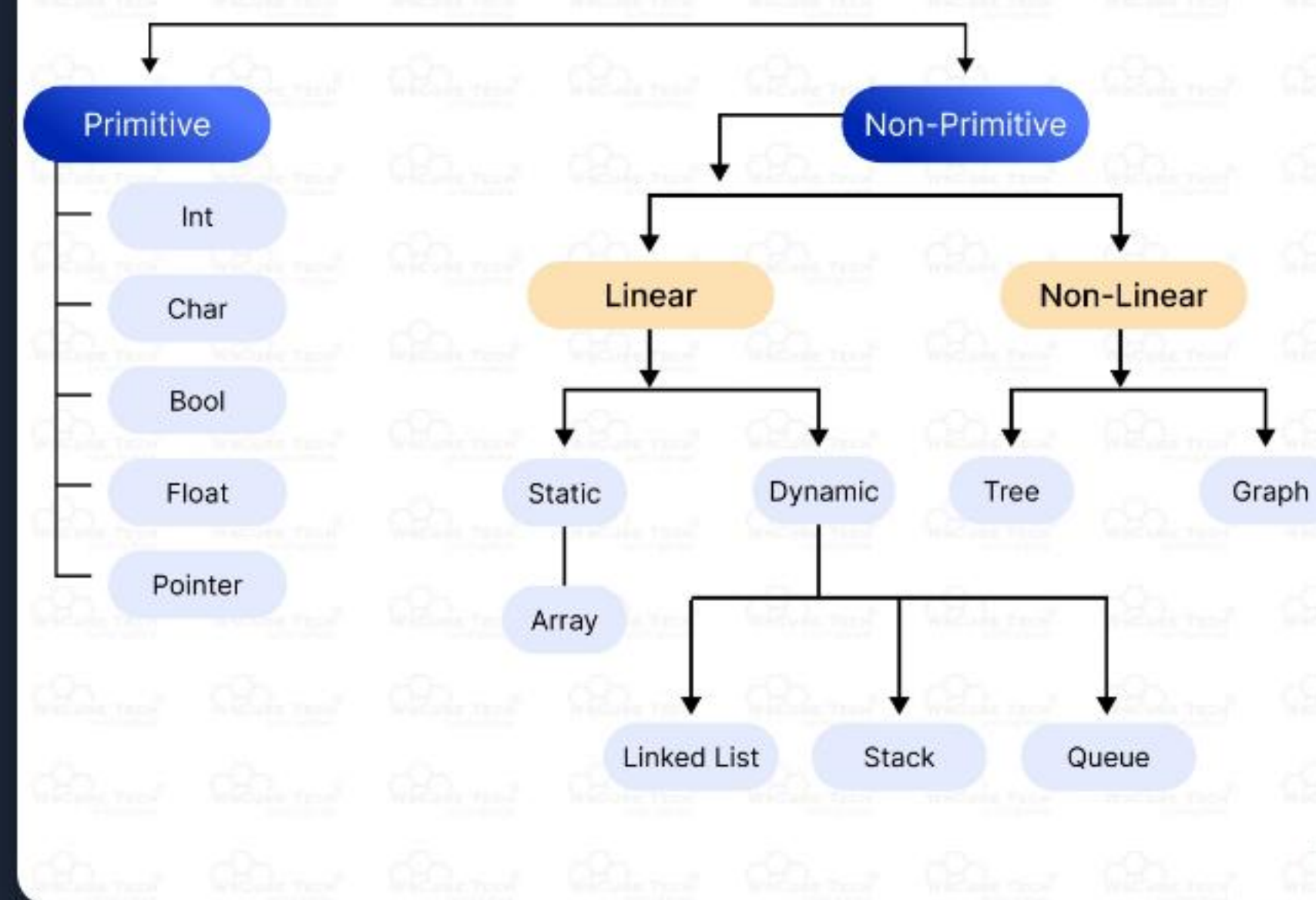


— DATABASE ARCHITECTURE PRINCIPLE

If you forget the primary key, proper data types, or NOT NULL constraints, your system will eventually become slow, messy, and hard to maintain. Professional design matters from Day 1.

Summary: What You Learned Today

Data Structure



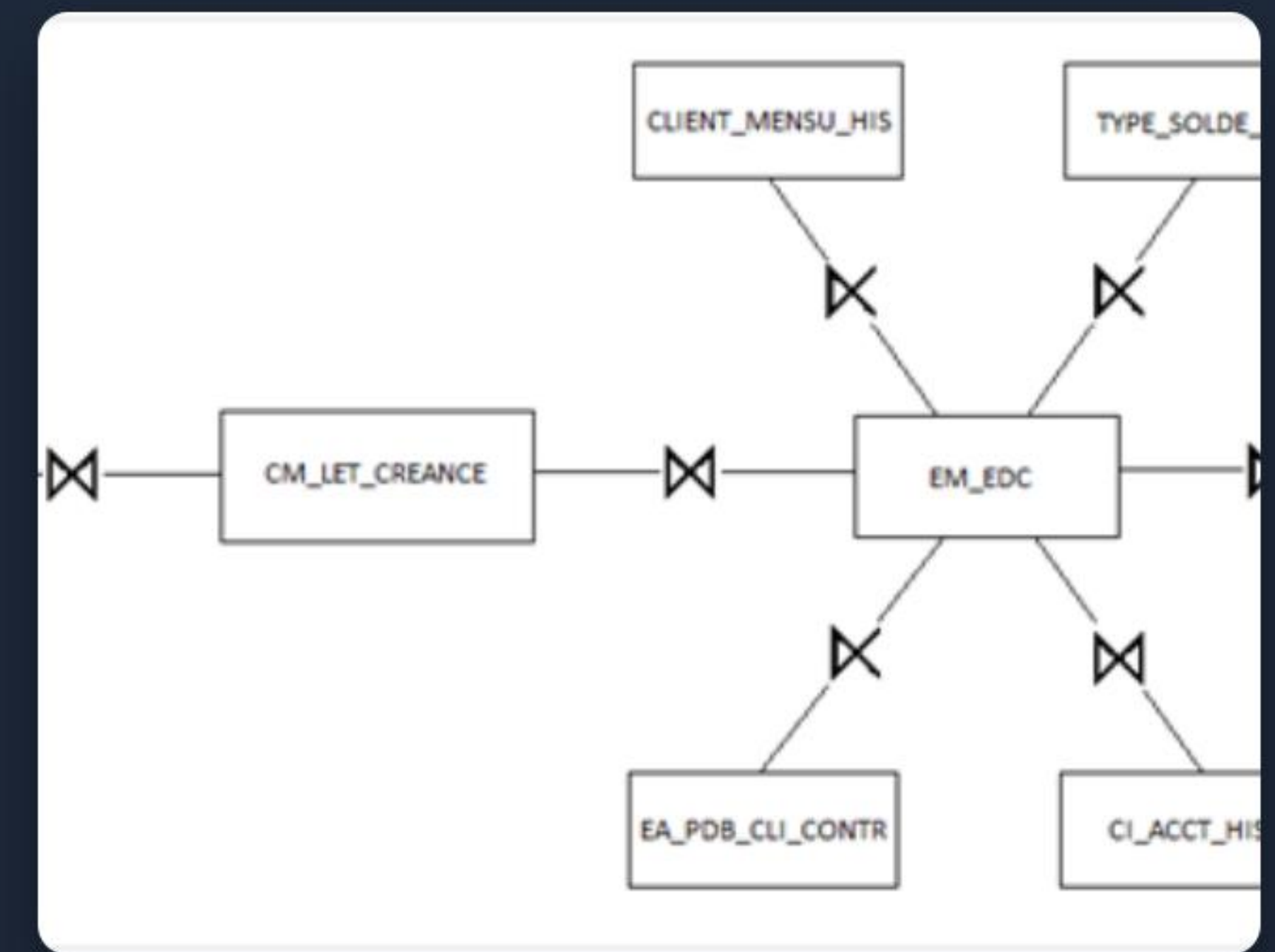
Data Types

INT, VARCHAR, FLOAT, DATE



Constraints

PRIMARY KEY, NOT NULL,
AUTO_INCREMENT



SQL Commands

CREATE, INSERT, SELECT

Image Sources



https://media.easy-peasy.ai/27feb2bb-aeb4-4a83-9fb6-8f3f2a15885e/2194894d-0508-44d3-8c16-dec55b40eab8_medium.webp

Source: easy-peasy.ai



https://chester.ac.uk/media/media/photography/photography/computer-science/AdobeStock_413020895-6240x4160.jpeg

Source: www.chester.ac.uk



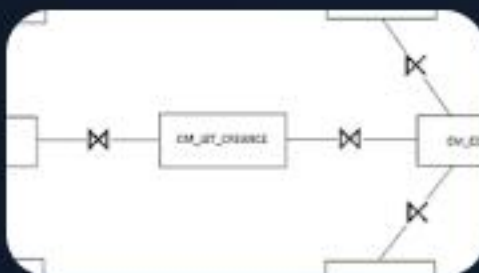
<https://deen3evddmddt.cloudfront.net/uploads/content-images/data-structure-classification.webp>

Source: www.wscubetech.com



https://png.pngtree.com/png-clipart/20240709/original/pngtree-d-lock-icon-cyber-security-digital-data-network-protection-future-technology-png-image_15523015.png

Source: pngtree.com



<https://i.sstatic.net/tMQM7.png>

Source: stackoverflow.com